Homework 4 - Dynamic Programming

Course: CIS 387 - Analysis of Algorithms - Fall 2026
Instructor: Dr. Aaron Mininger

Submission Instructions

Due Date: Nov 5 at 11:59pm

Zip up the 2 java files and upload to Canvas. **Your name must be at the top of each java file**

Note: You may not use any AI assistance or direct outside help on this assignment.

For 2 extra credit points, answer the following questions (leave a comment on Canvas):

1. Give an estimate of how much time you spent on this assignment.
2. How difficult did you find this assignment?
3. Was anything unclear, or is there any other feedback you would give?

Your responses will be helpful feedback for refining these assignments in future courses.

Grading Information

Points	Description
20	Effort: Does the program compile and run? Was a reasonable attempt made?
20	Part 1: Correct **`MakeChange.java`** program.
10	Part 1: **`MakeChange`** correctly uses bottom-up DP.
30	Part 2: Correct **`AntPath.java`** program.
10	Part 2: **`AntPath`** correctly uses top-down DP.

Setup

All the code and files needed for this homework are available on the course git repository: **`https://github.com/amininger-mu/CIS387F26Code`**

You will see starter files for each part that you should fill in. In addition, there are test files you can use to test each part (you should make additional test files for larger cases!).

1. Making Change

Consider the problem of making change for a given dollar amount with the fewest number of coins.

For example: what is the fewest number of coins needed to make change for 97 cents?
Answer: 7 coins (3 quarters, 2 dimes, and 2 pennies)

With US coins, you can use the greedy solution of always choosing the largest coin possible. But for arbitrary coin values, this isn't always true.

For example: if you had 3 types of coins worth 1, 7, and 10 cents, the optimal way to make 15 cents is 3 coins (1+7+7) and not using the 10 at all.

Directions:

You are given a file **`MakeChange.java`** which has some starter code to read a test case from stdin.

Implement the method **`int fewestCoinsNeeded(int amount, int[] coins)`** using a *bottom-up dynamic programming* solution to return the fewest number of coins needed to make the given amount, where **`coins`** is an array of possible coin values.

Your solution should not use any recursion. Refer to the code examples in **`lesson20`** (e.g. **`DivideRod.java`** and **`TrickOrTreat.java`**) as well as the slides for guidance.

Testing:

The **`MakeChange.java`** main function will read in the test case from standard in as:

- 1 int: the amount of cents to make change for
- 1 int: the number of different types of coins
- **`n`** ints: one for each coin value

Example: **`change_test1.txt`** (97 cents using US coins):

```
97
4
1 5 10 25
```

You can feed in a test case using:

```
cat change_test1.txt | java MakeChange.java
```

I have provided 2 test case files, but you should make others to test your algorithm.

Extra Credit

Modify your program so that it prints a list of coin values in the optimal solution. (Hint: use a similar technique as shown for the rod example in class).

Example: for **`change_test1.txt`**, it should print something like:
`To make 97 cents: 1 1 10 10 25 25 25`

2. Max Weight Path

Imagine an ant on a 2D grid of width *W* and height *H*. It starts at position (W-1, H-1) in the bottom-right corner, and needs to get to its anthill in the top-left corner (0, 0). At each step, it can only move 1 square north or west.

	2	3	1	2
2	6	1	2	1
5	4	4	3	2
2	1	1	5	

Every square has some amount of food (given as an integer). What is the maximum amount of food that the ant can eat in crossing the grid?

Here are two possible paths, the one on the right is an optimal solution:

	2	3	1	2
2	6	1	2	1
5	4	4	3	2
2	1	1	5	

Total Food: 13

	2	3	1	2
2	6	1	2	1
5	4	4	3	2
2	1	1	5	

Optimal Solution: 24

Directions:

You are given a file **`AntPath.java`** which has some starter code to read a test case from stdin.

Write a recursive solution that uses *top-down dynamic programming* (memoization) to compute the maximum food the ant can get on its journey. The base function should look like **`int bestPath(int[][] grid)`**, which should then call a recursive version.

Hint: With top-down dynamic programming, you pass around a lookup table (cache) keyed on the current state. Use the given **`Coord`** record structure to represent the state (current row/column). By using the Java **`record`**, it automatically creates a hash function and can be used as a key.

Testing:

The **`AntPath.java`** main function will read in the test case from standard in as:

1. 2 ints: the width and height of the grid
2. w*h ints: the amount of food on each square

Example: **`ant_test1.txt`**:

```
5 4
0 2 3 1 2
2 6 1 2 1
5 4 4 3 2
2 1 1 5 0
```

You can feed in a test case using:

```
cat ant_test1.txt | java AntPath.java
```

I have provided 1 test case file, but you should make others to test your algorithm.